How DNS Resolution Works: Resolvers, Records, TTL

why half your users still hit the old server

Watch the 4-min video

> **In one line:** your machine asks a recursive resolver, which walks root → TLD → authoritative on a cache miss; every answer is cached for its TTL, so lower the TTL before a migration and changes spread in minutes instead of hours.

1 The problem: DNS remembers

You move `api.example.com` to a new load balancer and update DNS. An hour later, half your users are still hitting the old one. Nothing is broken. It is DNS caching, doing exactly what you told it to. To control it, you need to know how a lookup really works.

DNS turns names into IP addresses. Your laptop does not search the internet itself. It asks a **recursive resolver**, usually run by your ISP, your cloud provider (inside a VPC, the provider's resolver), or a public service like `1.1.1.1` or `8.8.8.8`. The resolver does the hard work for you, and **caches** the answers so the next user gets them instantly.

2 The lookup, step by step

On a **cold cache**, the resolver walks down the naming tree, from right to left in the name:

1. It asks a **root server**: who handles `.com`? The root points it to the `.com` **TLD** (top-level domain) servers.
2. It asks a TLD server: who handles `example.com`? The TLD points it to example.com's **authoritative name servers**.
3. It asks the authoritative server, which gives the real answer: the IP address of `api.example.com`.

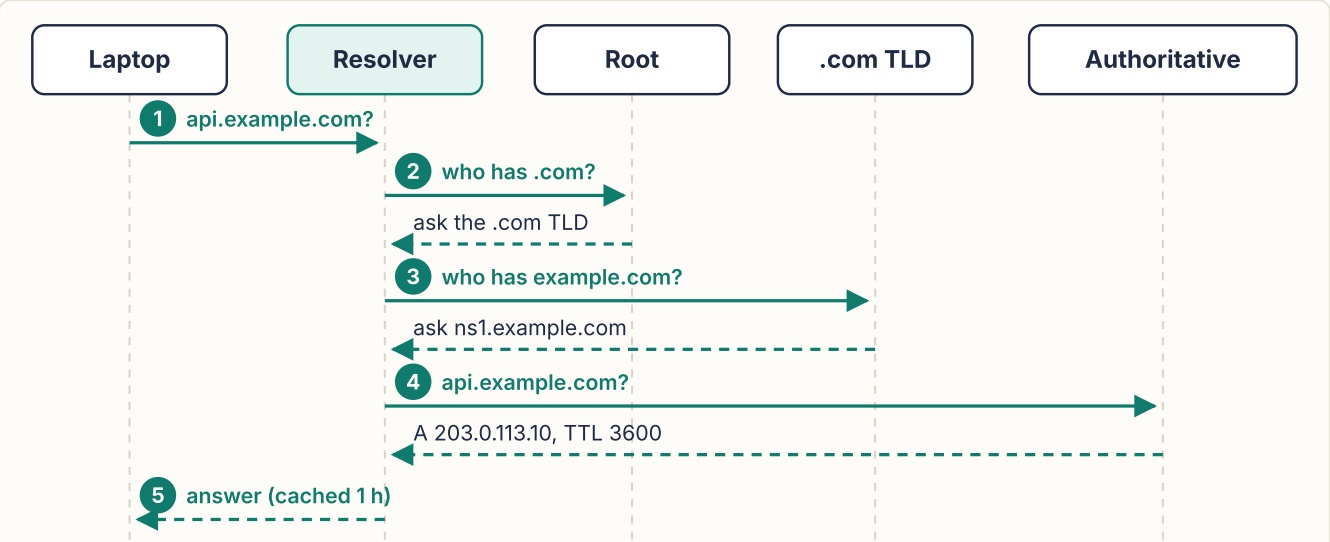


Figure 1. A cold-cache lookup. The laptop asks once; the resolver does every step and caches the answer for its TTL.

Root and TLD servers only *refer* the resolver onwards; only the authoritative server holds the records. On a warm cache most of this is skipped: the resolver already knows the `.com` and `example.com` servers, so a lookup is often one round trip, or zero.

3 Record types to know

Type	What it does	Example
A	Name → IPv4 address	`api.example.com A 203.0.113.10`
AAAA	Name → IPv6 address (“quad A”)	`api.example.com AAAA 2001:db8::10`
CNAME	Alias: points one name to another *name*, which is then resolved	`www CNAME my-lb.elb.amazonaws.com`
MX	Where email for the domain goes, with a priority	`example.com MX 10 mail.example.com`
TXT	Free text, often domain verification or email policy (SPF)	`example.com TXT "v=spf1 …"`
NS	Names the authoritative servers for a zone	`example.com NS ns1.example.com`

4 TTL and caching

Every record has a **TTL** (time to live) in **seconds**. Resolvers, operating systems and browsers all cache the answer until it expires. With a TTL of **3600**, a resolver may keep serving the old IP for **up to an hour** after you change it. (This DNS TTL is a cache lifetime; it has nothing to do with the hop counter called TTL in IP packets.)

Even failed lookups get cached. That is **negative caching**: if someone looked up a name before you created it, the “does not exist” answer is cached too (for a time set in the zone's SOA record), so the new name can keep failing for a while.

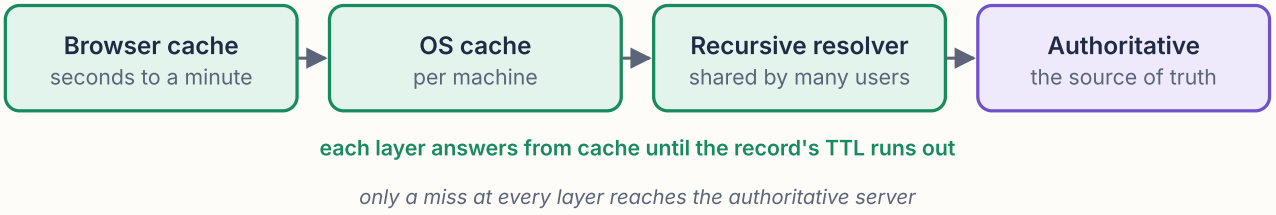


Figure 2. Answers are cached at several layers. A change reaches a user only when every layer between them and the authoritative server has expired its copy.

5 Migrating a record safely

Here is how to move `api.example.com` to a new load balancer with almost no stale traffic:

1. **A day or two ahead**, lower the TTL from 3600 to **60**.
2. **Wait at least one old TTL** (here, one hour), so every cache drops its 1-hour copy and picks up the short value.
3. **Switch the record** to the new load balancer. Within about a minute, most clients follow.
4. **Keep the old load balancer running** for a while (hours to days), because some clients and libraries ignore TTLs, and open connections stay where they are.
5. **Once stable, raise the TTL** again.

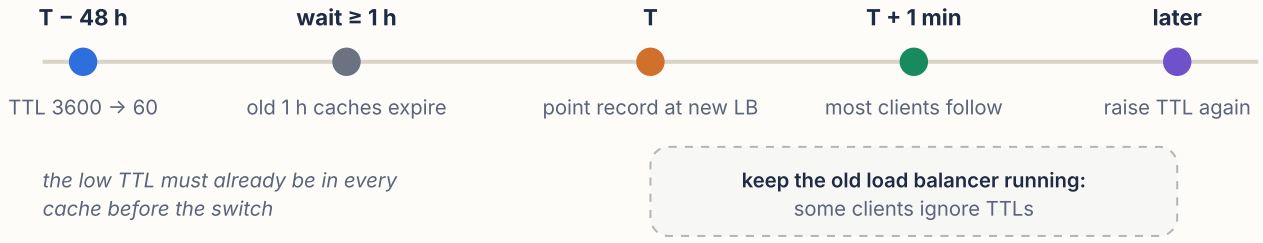


Figure 3. The safe migration. Lowering the TTL only helps once the old, long TTL has expired everywhere, so do it well before the switch.

WORKED EXAMPLE: WHAT GOES WRONG WITHOUT STEP 1

TTL is 3600. You switch the record at 10:00. A resolver that cached the answer at 09:59 keeps serving the old IP until **10:59**, and every user behind it hits the old load balancer for an hour. Lowering the TTL to 60 *at* 10:00 doesn't help them: they won't ask again until 10:59. Lower it to 60 the day before, and the worst case after the switch is about **60 seconds** for well-behaved caches.

WHY NOT KEEP TTLS LOW ALL THE TIME?

Short TTLs mean more lookups, more load (and cost) on DNS, and slightly slower first requests, because more of them miss the cache. Long TTLs are faster and cheaper, but changes take longer to spread. So use **long TTLs** (hours) for stable records and **short ones** (30–60 s) for records you may need to change quickly, such as a failover endpoint.

COMMON MISTAKES

- Lowering the TTL at the same moment as the change. The old, long TTL is still cached.
- Shutting down the old server as soon as DNS changes. Stale caches and long-lived connections still send traffic there.
- Testing from your own laptop and seeing a cached answer. Query the authoritative server directly (`dig @ns1.example.com api.example.com`).
- Treating a CNAME as an HTTP redirect. It is resolved inside DNS, before any connection is made.

6 Key terms

Recursive resolver: does the lookup for clients, caches answers

Root server: points to TLD servers

TLD server: points to a domain's name servers

Authoritative server: holds the real records for a zone

TTL: seconds an answer may be cached

Negative caching: caching "name does not exist"

A / AAAA: name → IPv4 / IPv6

CNAME: alias to another name

NS: which servers are authoritative

7 Interview questions

Q1. Walk through a DNS lookup. BASIC

The client asks a recursive resolver. On a cache miss, the resolver asks a root server (which refers it to the .com TLD), then the TLD server (which refers it to the domain's authoritative servers), then the authoritative server, which returns the record. The answer is cached for its TTL.

Q2. What's the difference between an A record and a CNAME? BASIC

An A record maps a name straight to an IPv4 address. A CNAME maps a name to another name, which must then be resolved; it's how you point `www` at a cloud load balancer whose IPs change.

Q3. What does a TTL of 3600 mean? BASIC

Any cache may keep serving that answer for up to 3,600 seconds (one hour) without asking again. A change can take up to an hour to reach users behind that cache, longer if a client ignores TTLs.

Q4. Why lower TTLs before a migration, and how far ahead? SCENARIO

So that when you switch the record, cached answers expire in minutes, not hours. Lower it at least one old TTL before the switch (a day or two ahead is common), or caches still hold the long value.

Q5. You updated DNS an hour ago and some users still hit the old server. Why? SCENARIO

Their resolver, OS or browser cached the old answer, either from a long TTL or because the client ignores TTLs; long-lived connections also stay put. Keep the old server up and draining until traffic to it stops.

Q6. You created a new subdomain, but it fails to resolve for some users for minutes. Why? SCENARIO

Negative caching. Someone queried it before it existed, and resolvers cached the “does not exist” answer for the time set in the zone's SOA record. It clears when that expires.

Q7. Why not set every TTL to 10 seconds? TRICKY

Far more lookups: more DNS load and cost, more cache misses, slower first requests, and more exposure if the DNS provider has an outage. Use short TTLs only for records you need to move fast.

Q8. Can you put a CNAME at the zone apex (example.com)? TRICKY

No. The apex must hold SOA and NS records, and a CNAME can't coexist with other records. Providers offer ALIAS or ANAME records (Route 53 alias records) that resolve the target and answer with A records instead.

Next: SD-N-016 · DNS part 2: records, TTL and safe migrations backendroadmap.com